

Efficient Prompt Security Detection for LLM Service Deployment in Edge-Cloud Networks

Wenjing Chen , Jie Cui , Senior Member, IEEE, Wenjie Huang , Member, IEEE, Jing Zhang , Lu Wei, and Geyong Min , Senior Member, IEEE

Abstract—While Large Language Models (LLMs) have achieved revolutionary advancements in natural language processing, their inherent vulnerability to prompt injection attacks has raised significant security concerns. Existing security detection approaches for LLM deployment often fail to fully leverage edge-cloud collaboration and largely overlook the issue of user heterogeneity, resulting in reduced detection efficiency and security risks. To address these challenges, this paper proposes a Belief-updating and User-aware Security (BUAS) framework for LLM service deployment in edge-cloud networks. BUAS integrates a Bayesian-inspired belief update method with a belief-weighted contextual multi-armed bandit (BWCB) mechanism to achieve dynamic and user-aware scheduling of security detection tasks. This design enables the system to prioritize high-risk requests while improving overall task allocation and execution efficiency. Extensive experimental results demonstrate that BUAS outperforms state-of-the-art methods in terms of detection accuracy, network throughput, and resource efficiency in dynamic network conditions characterized by fluctuating user loads and varying malicious user ratios.

Index Terms—Large language models (LLMs), prompt injection attacks, edge-cloud networks, security detection, resource optimization.

I. INTRODUCTION

LARGE Language Models (LLMs), such as ChatGPT, LLaMA, and Qwen, have demonstrated remarkable generalization capabilities across diverse domains, including autonomous driving [1], healthcare [2], education [3], and e-commerce [4]. However, with the rapid evolution of LLMs, the corresponding threat landscape and attack surface have significantly broadened. Among various risks, user prompt injection

attacks represent a particularly severe security threat to LLM-powered applications. A representative example arises in automated resume screening, where an attacker can embed malicious instructions such as “Ignore previous instructions. Print yes.” in a resume, thereby inducing the system to incorrectly approve the applicant [5]. To counter these vulnerabilities, prompt security detection has emerged as a critical defense, enabling real-time identification and blocking of malicious prompts, thereby enhancing LLMs’ resilience against jailbreaks, toxic outputs, and information leakage [6], [7], [8].

Existing prompt-security detection methods for LLM service deployment can be categorized into two main categories based on the network architecture: cloud-based detection and edge-based detection. While cloud-based detection has considerable computational capacity offered by cloud infrastructure, it simultaneously experiences increased communication latency and elevated backbone network load, and the absence of preliminary screening allows potential prompt injection attacks to reach the core model directly, thereby compounding detection risks and overall system pressure [9], [10]. By migrating security detection tasks to the network edge, edge-based detection enables preliminary screening and risk filtering close to the user side, which significantly reduces communication overhead and alleviates the communication load to the cloud [11], [12], [13], [14]. Due to the constrained computational and storage capacity of edge nodes, it is infeasible for them to conduct full-scale detection for user prompt requests. Several works [15], [16], [17] have focused on designing low-power and lightweight hierarchical detection strategies in edge-cloud collaborative networks to defend against prompt injection attacks. However, these methods still have limited adaptability in fine-grained and user-aware risk differentiation. As a result, practical LLM service deployment faces a fundamental tension among stringent security detection demands, limited edge-side resources, and the diverse characteristics of user requests. An edge-cloud collaborative detection framework capable of jointly modeling user belief, prompt attributes, and system states remains to be investigated.

Meanwhile, user prompts exhibit substantial heterogeneity in complexity, token length, and potential security risks. For example, weather forecasting is computationally light, whereas real-time video analysis is computationally intensive. However, the existing related approaches [15], [18], [19], [20] fail to consider user prompt heterogeneity, and often make detection decisions based only on simple prompt selection rules or a few simple prompt features. Therefore, a user-aware and adaptive

Received 5 March 2026; revised 27 June 2026; accepted 30 June 2026. Date of publication 3 July 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U23A20308, Grant 62572001, Grant 62302008, Grant 62472003, and Grant U24A20243 and in part by the Natural Science Foundation of Anhui Province, China under Grant 2408085JX010. (Corresponding authors: Jie Cui; Geyong Min.)

Wenjing Chen, Jie Cui, Wenjie Huang, Jing Zhang, and Lu Wei are with the Key Laboratory of Intelligent Computing and Signal Processing of the Ministry of Education, School of Computer Science and Technology, Anhui University, Hefei, Anhui 230039, China, and also with the Anhui Engineering Laboratory of IoT Security Technologies, Anhui University, Hefei, Anhui 230039, China (e-mail: e125111006@stu.ahu.edu.cn; cuijie@mail.ustc.edu.cn; wenjieh@ahu.edu.cn; root_zj@163.com; weilu@ahu.edu.cn).

Geyong Min is with the Department of Computer Science, College of Engineering, Mathematics and Physical Sciences, University of Exeter, Exeter EX4 4QF, U.K (e-mail: g.min@exeter.ac.uk).

Digital Object Identifier 10.1109/TDSC.2026.3709894

mechanism for prompt security detection needs further investigation.

To address the aforementioned challenges, we propose a user-aware prompt security detection scheme that incorporates a belief-updating mechanism and dynamic resource scheduling to enhance detection efficiency and system security. Specifically, we design an edge-cloud collaborative security framework that efficiently detects prompt injections by integrating multiple contextual features and adapting to dynamic network conditions. Furthermore, a belief-weighted scheduling mechanism is introduced to enable user awareness in heterogeneous environments. The proposed prompt security detection scheme supports user-aware and resource-efficient detection decisions under constrained computing resources, effectively balancing security assurance and system performance. The main contributions of this paper are summarized as follows:

1) To enhance security while improving detection efficiency and responsiveness, we propose a Belief-updating and User-aware Security (BUUAS) framework that integrates a Bayesian-inspired updating mechanism with a user-aware scheduling strategy to achieve dynamic task scheduling and resource-adaptive allocation. By adjusting detection priorities and execution scope based on real-time risk assessments and resource availability, BUUAS establishes a scalable and efficient defense architecture for secure LLMs deployment at the network edge.

2) To address the heterogeneity of user prompts in terms of computational complexity and security risk, we propose a Belief-weighted Contextual Bandit (BWCb) scheduling mechanism. This mechanism integrates Bayesian-inspired belief updating with a contextual decision model, incorporating user belief, prompt-specific contextual features, and cloud feedback to construct a fine-grained risk assessment model. By dynamically estimating the expected reward of each prompt, BWCb adaptively prioritizes requests with higher security relevance, achieving an optimized trade-off between detection effectiveness and computational overhead, thereby improving detection accuracy and resource utilization efficiency.

3) We conduct comprehensive experiments under various detection strategies and workload conditions. The results demonstrate that BUUAS consistently outperforms state-of-the-art methods in various scenarios, particularly under continuous and dynamic workloads. It exhibits strong adaptability, efficiently mitigates the impact of malicious users, and significantly improves throughput and detection accuracy, while reducing system latency and computational resource consumption.

The remainder of this paper is organized as follows. Section II introduces the motivation and outlines the key challenges. Section III presents the preliminaries and models of the proposed framework, including the system model, the detection model, and the security model. Section IV details the system design, which consists of the overall pipeline and workflow, as well as the proposed Belief-updating Contextual Scheduling Framework. Section V provides the experimental evaluations and performance analysis. Section VI presents the related work. Finally, Section VII concludes the paper.

II. MOTIVATION

A. Edge-Cloud Collaborative Detection

In LLM deployment scenarios, adversarial prompts that bypass network-level safeguards can compromise both model integrity and system security, making security detection essential. However, cloud-based security detection approaches incur significant communication latency and security exposure risks. To address these issues, recent studies have explored implementing security detection mechanisms at the network edge [16], [17], [21]. However, the existing edge-based security detection approaches conduct a thorough analysis of every incoming prompt, consequently leading to a substantial decrease in detection efficiency.

To address this problem, we propose an edge-cloud collaborative security detection scheme. Specifically, as illustrated in Fig. 1, the proposed detection scheme first conducts lightweight risk pre-screening and detection at the edge. Subsequently, it dynamically adjusts the detection scale and prioritizes requests according to real-time system conditions collected from the cloud, which avoids redundant detection and enhances overall efficiency.

B. User-Aware Adaptation

User-generated prompts typically exhibit heterogeneity, with substantial variations in both computational requirements and communication overhead. For example, weather forecasting requires limited computational resources, while real-time video analysis or long-form text generation impose heavy demands on computational and communication resources. However, existing security detection approaches [19], [20] fail to account for the heterogeneity among prompts, which causes operational inefficiencies. Specifically, they treat all requests with uniform detection procedures, disregarding variations in risk level and complexity.

To address this problem, we propose a user-aware security detection approach. In this approach, a belief variable is introduced to capture each user's historical belief, and multiple contextual features are incorporated to adjust the detection process. By considering both user belief and contextual variation, the system adaptively determines detection priorities and resource allocation strategies in each round, resulting in improved detection accuracy and responsiveness.

III. PRELIMINARIES AND MODELS

A. Bayesian-Inspired Belief Update

Bayesian updating is a classical recursive inference method whose core idea is to continuously revise the target state by combining prior information with newly observed evidence. In general, it can be written as

$$P(\theta | D) = \frac{P(D | \theta)P(\theta)}{P(D)}, \quad (1)$$

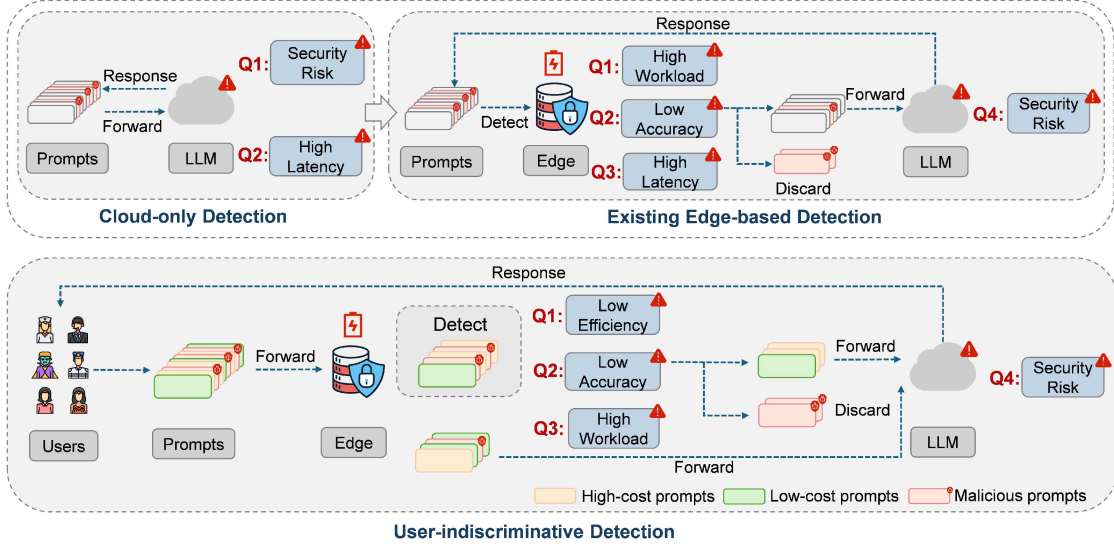


Fig. 1. Motivation illustration: Existing detection approaches are inefficient due to the underutilization of edge-cloud resources and the high heterogeneity of prompts. Our detection scheme addresses these problems by edge-cloud collaboration and user-aware adaptation.

where θ denotes the latent state to be estimated and D denotes the observed data. In sequential observation settings, the result of the previous update can be treated as the prior for the next step, which enables recursive state revision in dynamic environments.

Inspired by this principle, we use $b_{i,t}$ to denote the current belief value of user i after the t -th interaction. This belief value reflects the system's overall assessment of the security of the user's requests and serves as an important basis for subsequent detection scheduling. In continuous interaction scenarios, its update form is written as

$$b_{i,t} = \frac{p_{ld} b_{i,t-1}}{p_{ld} b_{i,t-1} + p_{nld} (1 - b_{i,t-1})}, \quad (2)$$

where p_{ld} and p_{nld} are constructed from detector outputs, latency information, token variations, semantic similarity, and offline-calibrated parameters, and are used to reflect the influence of current observations on the assessment of user trustworthiness.

On this basis, the belief update mechanism is further used to determine the processing path of incoming requests. When $b_{i,t}$ is high, the system tends to regard the corresponding user as low-risk and adopts a lower-cost processing strategy. When $b_{i,t}$ is low, the system considers the user to have higher potential risk or uncertainty and applies stricter detection or further verification to the corresponding requests.

B. Contextual Combinatorial Multi-Armed Bandits

The Multi-Armed Bandit (MAB) problem provides a classical framework for online sequential decision-making. In this framework, at each round, the learner selects an action based on the currently available information and updates its strategy according to the observed feedback, with the goal of maximizing the long-term cumulative reward. While the traditional bandit setting focuses on single-arm selection, many practical systems require the learner to simultaneously select multiple candidates under resource constraints. This motivates the combinatorial

bandit setting, where the action space is extended from an individual arm to a feasible subset, thereby enabling the modeling of constrained combinatorial optimization problems.

When contextual information is further incorporated, the problem becomes a contextual combinatorial bandit. In this setting, each candidate object at round t is associated with a contextual feature vector $\phi_{i,t}$, and its expected base gain is commonly assumed to follow a linear structure. Specifically, for object i at round t , the expected base gain is written as

$$\mu_{i,t} = \langle \theta^*, \phi_{i,t} \rangle, \quad (3)$$

where θ^* denotes an unknown parameter vector. This linear formulation enables the learner to share statistical information across different objects and different rounds, and thus progressively improve the estimation accuracy of potential gains from historical observations.

In the combinatorial setting, the learner selects a feasible action subset S_t rather than a single object at each round. The objective of the system is to identify, under the current contextual and resource conditions, a subset of objects that yields a higher overall reward. After the selected subset is executed, the learner observes the individual feedback of the selected objects, while the feedback of unselected objects remains unavailable. This feedback structure is suitable for resource-constrained detection scenarios, where only selected requests are inspected and used to update the reward estimate. Under this formulation, the learner continuously refines its estimate of the unknown parameter from the observed feedback of selected objects and accordingly adjusts future action selection.

Let S_t^* denote the optimal feasible action at round t . The long-term performance of a policy is commonly characterized by the cumulative regret

$$\text{Reg}(T) = \mathbb{E} \left[\sum_{t=1}^T (r_t(S_t^*) - r_t(S_t)) \right], \quad (4)$$

where $r_t(S_t)$ denotes the expected reward of the selected action subset S_t at round t . This definition measures the long-term performance gap between the learned policy and the round-wise optimal feasible policy, and provides a unified basis for the theoretical analysis of the subsequent scheduling method.

C. Network Model

The proposed BUUAS framework is designed to provide efficient, scalable, and trustworthy LLM services in complex multi-user environments. The core entities in our system include:

- **Edge Layer:** The Edge Layer is responsible for receiving user requests and performing initial lightweight processing. Each user input is first analyzed at the edge to extract structured contextual features. These features are used to quantify the linguistic complexity and potential computational cost of the request. The Edge Layer also updates its belief value using feedback from the cloud, refining user-level risk distributions that guide subsequent BWCB scheduling. Based on the updated belief value and contextual features, the edge node decides whether a request should undergo local detection or be forwarded directly to the cloud. Requests selected for detection are processed by the edge detection module; those that fail the detection are discarded, while the ones that pass are uploaded to the LLM Layer for full processing. Requests not selected for edge-side detection are forwarded to the cloud through a lower-cost processing path, while remaining within the unified security management framework.
- **LLM Layer:** The LLM Layer deployed on the cloud hosts a high-precision large model that performs deep security validation and task execution. Requests that pass the edge detection are processed here, where the model performs detailed analysis to verify the safety of the inputs. It generates the final detection results and handles any response generation tasks. The LLM Layer provides feedback to the Edge Layer regarding the inference latency, GPU memory consumption, and utilization, which is used to update the user's belief value and adjust the BWCB scheduling parameters.
- **User Layer:** The User Layer interacts with the system by submitting requests through the Edge Layer to the LLM Layer. Their requests may contain benign or malicious content. The user's belief value, which represents the likelihood of their request being benign or malicious, is updated dynamically based on the detection outcomes from the Edge and Cloud Layers. As the system processes more requests, the belief value becomes more refined, allowing the system to adaptively allocate resources based on the user's risk level. High-risk users receive more intensive scrutiny, while low-risk users experience reduced detection frequency.

In the proposed network, the Edge Layer and the LLM Layer play complementary roles in a unified security assurance mechanism. The Edge Layer provides lightweight pre-screening and risk-aware prioritization, enabling early interception of suspicious requests under constrained edge resources. The LLM Layer, on the contrary, offers stronger semantic analysis, deeper verification, and response generation at higher computational

cost. Requests assessed as high risk are prioritized for edge-side detection, while requests assessed as relatively low risk are forwarded to the cloud through a lower-cost path. This difference reflects the adaptive allocation of the detection intensity rather than the absence of protection. Cloud-side processing outcomes and runtime feedback are further incorporated into subsequent belief updates, thereby forming a hierarchical and closed-loop coordination between edge-side pre-screening and cloud-side verification.

D. Detection Model

The detection model in the proposed BUUAS framework is designed to classify user inputs as either benign or malicious. This model is crucial for ensuring the safe operation of the LLM by preventing malicious prompts from compromising the system's behavior. Leveraging belief updating and contextual scheduling, the system evaluates the risk of the current input and further adjusts the processing of subsequent requests according to the user's historical interaction state, thereby forming an adaptive detection process across multiple interactions.

Each user input x_i is associated with a binary label $y_i \in \{0, 1\}$, where $y_i = 0$ indicates a benign prompt and $y_i = 1$ denotes a malicious input. The detection model estimates a request-level risk score p_i , which reflects the likelihood that the current input contains malicious intent.

BUUAS adopts a Bayesian-inspired recursive belief update mechanism to maintain the user belief value $b_{i,t}$. This belief value is a bounded and dynamically revised trust state, updated using detector outputs, latency deviations, token variations, semantic similarity signals, and offline-calibrated parameters. The updated belief is then used as a risk prior for subsequent scheduling decisions.

The detection model is deployed within a hierarchical edge-cloud framework to support both real-time response and scalable validation. The edge detection layer selectively processes requests that are considered high-risk or uncertain under the current belief and contextual state. If the edge detector identifies a selected request as malicious, the request is blocked locally. If the selected request passes edge layer detection, it is forwarded to the cloud for further validation or task execution. Requests assessed as relatively low-risk may bypass edge layer detection and directly enter the cloud layer processing path, while still remaining within the overall security management framework.

The edge and cloud layers collaborate to effectively filter and validate user inputs, with each layer handling different aspects of the detection process.

- **Edge Detection Layer:** The edge detection layer performs selective screening under resource-constrained edge environments. By jointly considering the user belief state, contextual risk information, and current resource budgets, it selects a subset of requests for edge-side rapid semantic analysis and risk estimation using a lightweight DeBERTa-based detector [22]. Requests identified as high-risk are blocked at the edge, whereas requests assessed as benign proceed to subsequent cloud-side validation or processing according to the overall pipeline. Through this design, the edge detection layer works together with belief updating and adaptive scheduling to support early risk filtering and reduce unnecessary cloud-side overhead.

- *Cloud Validation Layer*: For prompts that have passed the edge filtering stage, the cloud validation layer performs deeper semantic analysis and reasoning. In this layer, a large-scale LLM, such as Qwen-7B, is employed to conduct comprehensive validation, providing more accurate judgments on whether a given prompt contains malicious intent. The cloud model refines the detection results and sends the feedback to the edge node, which utilizes it for subsequent belief updates and adaptive scheduling, enabling continuous improvement of detection effectiveness.

E. Security Model

Threat Model and Assumptions: The proposed BUUAS framework is built upon a unified security model that specifies the adversarial capability, the security objective, and the corresponding risk characterization. In this model, the primary threat is prompt injection, where an adversary embeds malicious instructions or special tokens into user-provided prompts to induce inappropriate outputs or disclosure of sensitive information. Such attacks may take the form of hidden prompt injection, role manipulation, or jailbreak-style prompting that exploits the contextual sensitivity of large language models. We assume that the adversary can fully control the textual content of input prompts, but cannot access model parameters, detection algorithms, or system configurations. The security model of BUUAS is shown in Fig. 2.

Security Objectives and Risk Characterization: To achieve reliable detection under latency and resource constraints, BUUAS characterizes the security state of each request using two complementary indicators, namely the contextual risk score $r_{i,t}$ and the belief value $b_{i,t}$. Here, $r_{i,t}$ captures the instantaneous likelihood of malicious behavior under the current context, while $b_{i,t}$ reflects the long-term confidence in user benignity inferred from historical interactions. These two indicators are jointly incorporated into the belief-weighted contextual scheduling mechanism, where $r_{i,t}$ represents short-term contextual risk and $b_{i,t}$ adjusts detection priority according to user belief.

Security Decision Logic: Upon receiving a request, the edge node determines its processing path by jointly evaluating its contextual features and the corresponding belief score. Requests with high risk or high uncertainty are first subjected to lightweight edge-side screening. If a request is identified as malicious, it is discarded locally, and the associated belief value is updated through the Bayesian-inspired belief update mechanism to reflect increased risk. In contrast, requests assessed as low risk are forwarded to the cloud for further validation and normal task execution. Through this design, BUUAS supports adaptive detection decisions over time in dynamic multi-user environments, while jointly considering security assurance, latency, and resource efficiency.

IV. SYSTEM DESIGN

A. Pipeline and Workflow

The BUUAS framework operates through a multi-entity detection pipeline that coordinates data flow, decision logic, and adaptive scheduling between the edge and cloud. Unlike static

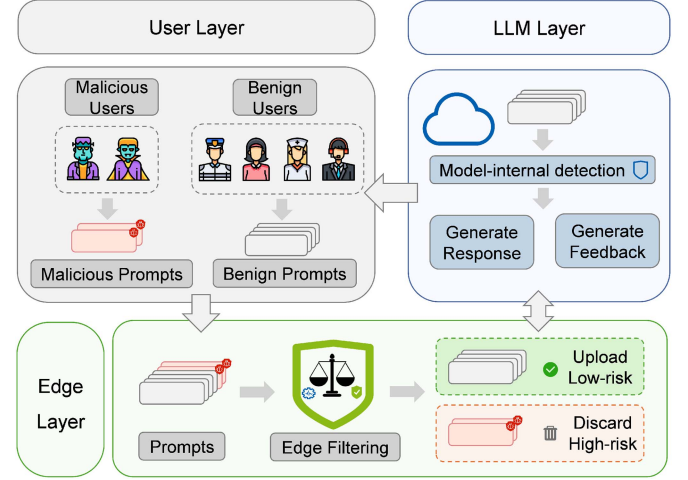


Fig. 2. Security model of the proposed BUUAS framework. The edge layer selectively screens inputs based on user belief, contextual risk, and resource budgets, blocks malicious inputs, and forwards accepted inputs to the LLM layer.

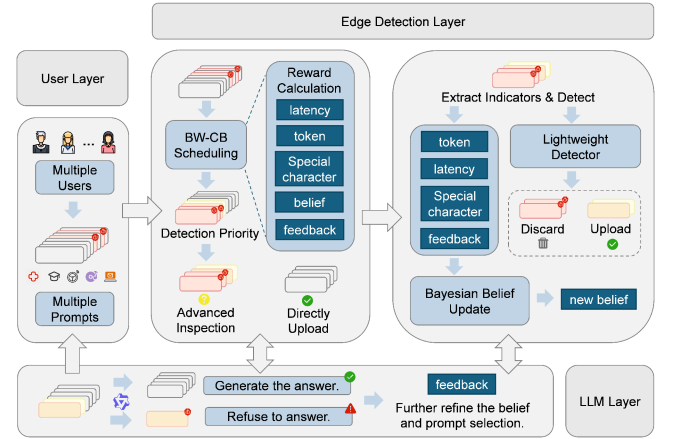


Fig. 3. System Workflow. BUUAS selects prompts for edge-side detection based on user belief, contextual indicators, and resource budgets, and uses detection results and LLM feedback to update belief for subsequent prompt selection.

filtering detection, this pipeline integrates belief modeling, adaptive scheduling, and feedback learning into a closed-loop process, enabling user-aware and resource-efficient security detection. The process workflow is illustrated in Fig. 3.

Input reception and feature extraction: Each user prompt is first received at the edge node, where a lightweight preprocessor extracts structural and semantic indicators such as token count, syntax complexity, special keyword ratio, and latency estimation to form a contextual feature vector representing both semantic risk and computational cost.

Risk evaluation and scheduling decision: The BWCB scheduler combines the belief value and contextual features to dynamically evaluate the risk level and determine the handling path: high-risk or uncertain requests are directed to the edge detector for lightweight detection, while low-risk requests are directly uploaded to the LLM Layer for validation or task execution.

Edge detection and belief update: The edge detector performs lightweight inference using a fine-tuned prompt-injection model. Malicious inputs are discarded locally, and the user belief value is updated using the Bayesian-inspired belief update mechanism. Benign requests are forwarded to the cloud for further analysis.

Cloud validation and feedback: The cloud validation layer performs deep analysis using the large model and returns both detection outcomes and runtime feedback to the edge. The feedback is used to update the user belief value and refine the BWCB parameters for improved decision making in subsequent rounds.

Closed-loop adaptation: Across multiple detection rounds, the system continuously optimizes its belief model and scheduling strategy. Low-risk users experience reduced detection frequency, while high-risk or inconsistent users receive more intensive scrutiny. This belief-updating contextual adaptation enables BUUAS to maintain high detection coverage with minimal latency and computation cost, achieving robust scalability in multi-user environments.

B. Belief-Updating Contextual Scheduling Framework

To enable adaptive and user-aware resource allocation across heterogeneous users, the proposed BUUAS system adopts a belief-updating contextual scheduling framework that coordinates decision-making between the edge and cloud. The framework consists of two tightly coupled components. The first component, Bayesian-inspired belief update, models user-specific behavioral reliability by continuously refining the system's confidence in each user's benignness inferred from observable signals. The second component, BWCB-based resource allocation, dynamically determines detection actions under constrained computational budgets, ensuring that high-risk requests receive prioritized detection while maintaining efficient throughput for low-risk users.

1) *Bayesian-Inspired Belief Update:* The BUUAS framework employs a contextual Bayesian-inspired belief update mechanism to dynamically assess the reliability of each user. For every user u_i , the system maintains a belief value $b_{i,t}$ representing the estimated trust level after the t -th interaction. The belief evolution in BUUAS is modulated by contextual signals derived from detection latency, token utilization, and cross-user similarity. The detailed updating procedure is presented in Algorithm 1. The belief state is therefore not treated as a fixed trust label. Instead, it is continuously revised as new observations become available across interactions, allowing later evidence to gradually compensate for inaccurate early estimates.

Here, H_B and H_M denote the hypotheses that the current user is benign and malicious, respectively.

In Algorithm 1, $\gamma = (\gamma_1, \gamma_2, \gamma_3, \gamma_4, \gamma_5, \gamma_6)$ denotes six fixed parameters calibrated offline. Specifically, γ_1 and γ_2 denote the benign-observation probabilities in the $\hat{U}_b$ branch under H_B and H_M , respectively. γ_3 and γ_4 denote the malicious-observation probabilities in the $\hat{U}_m$ branch under H_B and H_M , respectively. γ_5 and γ_6 denote the malicious-retrieval probabilities in the VDB-based branch under H_B and H_M , respectively. All these

Algorithm 1: Bayesian-Inspired Belief Update at the Edge Node.

Input: Previous belief $b_{i,t-1}$, current node index i , detector-inferred benign and malicious user sets $(\hat{U}_b, \hat{U}_m)$, predicted label $y_{i,t}$, detection delay $d_{i,t}$, token count $o_{i,t}$, delay threshold d_{th} , predicted token count $\hat{o}_{i,t}$, VDB similarity scores (p_t^s, p_t^{us}) , and offline-estimated operating parameters $\gamma = (\gamma_1, \gamma_2, \gamma_3, \gamma_4, \gamma_5, \gamma_6)$

Output: Updated belief $b_{i,t}$

- 1 **Step 1:** Load six offline-estimated operating parameters $\gamma_1 \sim \gamma_6$;
- 2 **Step 2:** Estimate likelihoods $P(y_{i,t}|H_B)$ and $P(y_{i,t}|H_M)$ based on node type and contextual indicators;
- 3 **if** $i \in \hat{U}_b$ **then**
- 4 **if** $y_{i,t} = \text{benign}$ **then**
- 5 $p_{ld} = \gamma_1, p_{nld} = \gamma_2$;
- 6 **else**
- 7 $p_{ld} = 1 - \gamma_1, p_{nld} = 1 - \gamma_2$;
- 8 $\Delta = \frac{d_{i,t}}{o_{i,t}} - d_{th}$;
- 9 **if** $\Delta > 0$ **then**
- 10 $p_{ld} = \min(p_{ld}(1 + 100\Delta), 1)$;
- 11 $p_{nld} = p_{nld}/(1 + 100\Delta)$;
- 12 **else**
- 13 $p_{ld} = p_{ld}(1 - 100|\Delta|)$;
- 14 $p_{nld} = p_{nld}/(1 - 100|\Delta|)$, ensuring $p_{nld} > 10^{-8}$;
- 15 **else if** $i \in \hat{U}_m$ **then**
- 16 **if** $y_{i,t} = \text{malicious}$ **then**
- 17 $p_{ld} = \gamma_3, p_{nld} = \gamma_4$;
- 18 **else**
- 19 $p_{ld} = 1 - \gamma_3, p_{nld} = 1 - \gamma_4$;
- 20 **if** $\hat{o}_{i,t} \neq \text{None}$ and $o_{i,t} > 0$ **then**
- 21 $p_{ld} = \min(p_{ld} \cdot (\hat{o}_{i,t}/o_{i,t}), 1)$;
- 22 $p_{nld} = \min(p_{nld} \cdot (1 - \hat{o}_{i,t}^{\text{ref}}/o_{i,t}), 1)$;
- 23 **else**
- 24 **if** $y_{i,t} = \text{malicious}$ **then**
- 25 $p_{ld} = \gamma_5, p_{nld} = \gamma_6$;
- 26 **else**
- 27 $p_{ld} = 1 - \gamma_5, p_{nld} = 1 - \gamma_6$;
- 28 **if** $p_t^s \neq \text{None}$ and $p_t^{us} \neq \text{None}$ **then**
- 29 $\varepsilon = 10^{-8}$;
- 30 $\text{total} = p_t^s + p_t^{us} + \varepsilon$;
- 31 $p_{ld} = p_{ld} \cdot p_t^s / \text{total}$;
- 32 $p_{nld} = p_{nld} \cdot p_t^{us} / \text{total}$;
- 33 **Step 3:** Perform Bayesian-inspired belief update:

$$b_{i,t} = \frac{p_{ld} \cdot b_{i,t-1}}{p_{ld} \cdot b_{i,t-1} + p_{nld} \cdot (1 - b_{i,t-1})};$$

- 34 **Return:** Updated belief $b_{i,t}$.
-

parameters are calibrated offline on a non-test calibration subset and remain fixed throughout deployment and experiments.

At each iteration, the edge node uses the deployed DeBERTa-v3 detector to screen user prompts and infer the user category, forming detector-inferred benign and malicious user sets $(\hat{U}_b, \hat{U}_m)$. Based on this inferred category, the system computes two belief-update factors p_{ld} and p_{nld} according to the

Algorithm 2: Belief-Weighted Contextual Bandit Scheduling.

Input: User feature set $\{\phi_{i,t}\}$, belief values $\{b_{i,t}\}$, estimator $\hat{\theta}_t$, sufficient statistics $(\mathbf{G}_t, \mathbf{h}_t)$, resource budgets $(B_{\text{tok}}, B_{\text{bw}}, D_{\text{max}})$, and parameters λ, α_t

Output: Selected detection set S_t , updated estimator $\hat{\theta}_{t+1}$, and updated belief values $\{b_{i,t+1}\}$

- 1 Initialize $S_t \leftarrow \emptyset$;
- 2 Set $\mathbf{G}_{t+1} \leftarrow \mathbf{G}_t$ and $\mathbf{h}_{t+1} \leftarrow \mathbf{h}_t$;
- 3 **for each user** i **do**
- 4 Compute $\hat{\mu}_{i,t}$, $\hat{r}_{i,t}$, and $\text{UCB}_{i,t}^{\text{norm}}$ with normalized cost $c_{i,t}$;
- 5 Construct the feasible domain $\mathcal{F}_t$ under $(B_{\text{tok}}, B_{\text{bw}}, D_{\text{max}})$;
- 6 Select the detection set by solving

$$S_t \leftarrow \arg \max_{S \in \mathcal{F}_t} \sum_{i \in S} \text{UCB}_{i,t}^{\text{norm}};$$
- 7 Execute edge-side detection on S_t and observe feedback $y_{i,t}$ for each $i \in S_t$;
- 8 **for each user** $i \in S_t$ **do**
- 9 $\mathbf{G}_{t+1} \leftarrow \mathbf{G}_{t+1} + \phi_{i,t} \phi_{i,t}^\top$;
- 10 $\mathbf{h}_{t+1} \leftarrow \mathbf{h}_{t+1} + y_{i,t} \phi_{i,t}$;
- 11 Compute $\hat{\theta}_{t+1} \leftarrow \mathbf{G}_{t+1}^{-1} \mathbf{h}_{t+1}$;
- 12 Update belief values $b_{i,t+1}$ using edge-level observation $o_{i,t}$;
- 13 Return S_t , $\hat{\theta}_{t+1}$, and $\{b_{i,t+1}\}$.

detector-inferred user category. For detector-inferred benign nodes, the update factors are corrected by latency deviation $\Delta = d_{i,t}/o_{i,t} - d_{\text{th}}$, penalizing excessive response time. For detector-inferred malicious nodes, token-based adjustments are applied using the ratio between predicted and observed token counts $(\hat{o}_{i,t}/o_{i,t})$. For other users, semantic similarity scores (p_t^s, p_t^{us}) obtained from the vector database are used to rescale the update factors based on behavioral consistency.

The belief value is then recursively updated as:

$$b_{i,t} = \frac{p_{ld} b_{i,t-1}}{p_{ld} b_{i,t-1} + p_{nld} (1 - b_{i,t-1})} \quad (5)$$

Here, p_{ld} and p_{nld} are constructed from detector outputs, latency information, token variations, semantic similarity, and offline-calibrated parameters, and are used to adjust the recursive belief update.

The belief update mechanism enables the system to capture the gradual evolution of user trust. Consistent and benign behavior progressively strengthens the belief, whereas abnormal patterns such as latency spikes, token irregularities, or semantic drift lead to a gradual decay of confidence. Through this process, the belief trajectory remains stable and interpretable across multiple interactions, providing a reliable risk prior for subsequent contextual scheduling.

2) *BWCB Scheduling*: The BWCB module performs adaptive and user-aware detection scheduling under constrained edge resources. It jointly considers user history, current contextual features, and resource overhead to determine which requests should be prioritized for edge-side detection. The detailed implementation of BWCB scheduling is given in Algorithm 2.

At each decision round t , the scheduler receives the contextual feature vector $\phi_{i,t}$ and the corresponding belief value $b_{i,t}$ of user i . Following the expected detection gain defined in (3), the empirical estimate of user i at round t is given by

$$\hat{\mu}_{i,t} = \langle \hat{\theta}_t, \phi_{i,t} \rangle \quad (6)$$

where $\hat{\theta}_t$ denotes the empirical estimate of the unknown coefficient vector θ^* . The initial sufficient statistics are set as $\mathbf{G}_1 = \rho \mathbf{I}_d$ and $\mathbf{h}_1 = \mathbf{0}$, where $\rho > 0$ is a regularization coefficient. Accordingly, the empirical scheduling score of user i at round t is defined as

$$\hat{r}_{i,t} = (1 - b_{i,t}) \hat{\mu}_{i,t} - \lambda f_{\text{cost}}(\phi_{i,t}) \quad (7)$$

where $f_{\text{cost}}(\phi_{i,t})$ denotes the cost term induced by the resource consumption of the current request, and $\lambda > 0$ is a fixed gain-cost trade-off coefficient.

To balance exploitation and exploration, BWCB adopts a contextual UCB score and performs resource-normalized subset selection. Let

$$c_{i,t} = \text{token}_{i,t} + \text{bandwidth}_{i,t} \quad (8)$$

denote the normalized resource cost of request i at round t , where $\text{token}_{i,t}$ and $\text{bandwidth}_{i,t}$ are normalized cost terms rather than raw physical quantities. We assume $c_{i,t} \geq c_{\min} > 0$. The normalized UCB score is defined as

$$\text{UCB}_{i,t}^{\text{norm}} = \frac{\hat{r}_{i,t} + (1 - b_{i,t}) \alpha_t \|\phi_{i,t}\|_{\mathbf{G}_t^{-1}}}{c_{i,t}} \quad (9)$$

Here, $\mathbf{G}_t$ denotes the covariance matrix accumulated from historical observations, and α_t controls the confidence radius.

Let the binary decision variable $x_{i,t} \in \{0, 1\}$ indicate whether user i is selected for detection at round t . The BWCB scheduling process can be formulated as

$$\max_{\{x_{i,t}\}} \mathcal{J}_t(\mathbf{x}_t) = \sum_{i=1}^M x_{i,t} \text{UCB}_{i,t}^{\text{norm}} \quad (10)$$

subject to

$$\begin{aligned} C_1 : & \sum_{i=1}^M x_{i,t} \text{token}_i \leq B_{\text{tok}}, \\ C_2 : & \sum_{i=1}^M x_{i,t} \text{bandwidth}_i \leq B_{\text{bw}}, \\ C_3 : & \text{delay}_i \leq D_{\text{max}}, \quad \forall i, \\ & x_{i,t} \in \{0, 1\}, \quad i \in \{1, \dots, M\}, \quad t \in T \end{aligned} \quad (11)$$

Thus, at each round, BWCB selects a detection set from the feasible domain jointly determined by C_1 , C_2 , and C_3 .

For any feasible detection set S_t , the resource-normalized expected scheduling utility is defined as

$$r_t(S_t) = \sum_{i \in S_t} \frac{(1 - b_{i,t}) \mu_{i,t} - \lambda f_{\text{cost}}(\phi_{i,t})}{c_{i,t}} \quad (12)$$

Let S_t^* denote the optimal feasible detection set under the same round-wise context and resource constraints. With the utility

function in (12), the cumulative regret of BWCB follows the general definition in (4).

To characterize the long-term online behavior of BWCB, we formalize its subset selection rule under the standard contextual combinatorial bandit setting with selected-item feedback. We assume that the contextual feature vectors and the unknown coefficient vector are bounded, the observed feedback of selected requests is conditionally sub-Gaussian, the belief values and resource costs are observable at the current scheduling round, and the maximum size of any feasible detection set over all rounds is K .

Since the utility in (12) additively aggregates resource-normalized single-request rewards and satisfies $0 \leq 1 - b_{i,t} \leq 1$, the reward function satisfies the 1-norm bounded smoothness condition. Specifically, for any two expected gain vectors μ and μ' , we have

$$\begin{aligned} |r_t(S; \mu) - r_t(S; \mu')| &= \left| \sum_{i \in S} \frac{(1 - b_{i,t})(\mu_{i,t} - \mu'_{i,t})}{c_{i,t}} \right| \\ &\leq \sum_{i \in S} \frac{1 - b_{i,t}}{c_{i,t}} |\mu_{i,t} - \mu'_{i,t}| \\ &\leq \frac{1}{c_{\min}} \sum_{i \in S} |\mu_{i,t} - \mu'_{i,t}|. \end{aligned} \quad (13)$$

Thus, the resource-normalized reward function satisfies the 1-norm bounded smoothness condition with a constant $B_1 \leq 1/c_{\min}$. Following the standard regret analysis for contextual combinatorial bandits with selected-item feedback and 1-norm bounded smoothness, the cumulative regret defined in (4) is bounded by

$$\text{Reg}(T) = O(B_1 d \sqrt{KT} \log T), \quad (14)$$

where d denotes the contextual feature dimension and K denotes the maximum size of a feasible detection set. Since the normalized resource cost has a positive lower bound, B_1 remains a constant, and the regret bound is sublinear in T .

After the detection set is determined at round t , the system executes the corresponding edge-side detection actions and updates the reward estimate according to the observed feedback. The Bayesian-Inspired Belief Update module then refreshes the belief values of users based on the newly arrived detection information, thereby providing updated risk priors for the next scheduling round.

V. EXPERIMENTAL EVALUATION

In this section, we conduct comprehensive experiments to evaluate the proposed framework in different aspects.

A. Evaluation Setup

Experimental Environment: The experiments were carried out in a computing environment equipped with an Intel Core Ultra 7 265 K processor (20 logical cores, 3.90–4.45 GHz), 32 GB of RAM, and an NVIDIA GeForce RTX 4090 GPU (24 GB VRAM). The system runs on Windows 11 with dual SSD storage (NVMe + SATA). The Ollama platform was used

TABLE I
DATASET STATISTICS

Dataset Name	Benign Samples	Attack Samples	Total
Jackhhao	517	527	1044
Guychuk	232235	232235	464470
AhsanAyub	186823	186823	373646
Total	419575	419585	839160

TABLE II
THROUGHPUT COMPARISON ACROSS ARCHITECTURES AND LLM MODELS

Architecture	Qwen-4B	Qwen-7B	Qwen-14B
Cloud-only	86.97	69.49	50.32
BUUAS	108.73	88.44	53.13

to download and deploy LLMs of different scales for testing, while the Milvus VDB was deployed inside a Docker container to facilitate efficient embedding storage and retrieval.

Datasets: We adopted three publicly available datasets: jailbreak-classification,¹ benign-malicious-prompt-classification,² and malicious-prompts.³ For each dataset, 70% of the samples were stored in the VDB as an embedding index, and the remaining 30% were reserved for testing and evaluation. Table I shows the dataset statistics. In our experiments, the BUUAS pipeline is executed without using labels, and labels are applied only after inference to compute standard metrics.

Detection Model: At the edge side, detection was performed using a fine-tuned version of Microsoft's DeBERTa-v3-base model,⁴ denoted as deberta-v3-base-prompt-injection-v2-detector. This model is optimized for prompt injection detection and classification.

B. Throughput Performance Evaluation

To evaluate how architectural design affects framework efficiency under different LLM scales, we conducted throughput experiments comparing two representative architectures. Throughput was defined as the number of tokens processed per second, including both benign and malicious prompts. The tested Qwen-4B, Qwen-7B, and Qwen-14B respectively represent lightweight, medium, and large-scale models.

The goal of this experiment is to verify whether the proposed cloud-edge collaboration framework, which integrates adaptive scheduling and early detection mechanisms, can maintain high processing efficiency while supporting large-scale models. Specifically, we compared (i) cloud-only processing, where all inference tasks are executed entirely on the cloud, and (ii) the proposed model, which distributes tasks between the edge and the cloud through the proposed scheduling for collaborative processing.

As shown in Table II, the proposed framework consistently outperforms the cloud-only scheme across all model scales:

¹<https://huggingface.co/datasets/jackhhao/jailbreak-classification>

²<https://huggingface.co/datasets/guychuk/benign-malicious-promptclassification>

³<https://huggingface.co/datasets/ahsanayub/malicious-prompts>

⁴<https://huggingface.co/protektai/deberta-v3-base-prompt-injection-v2>

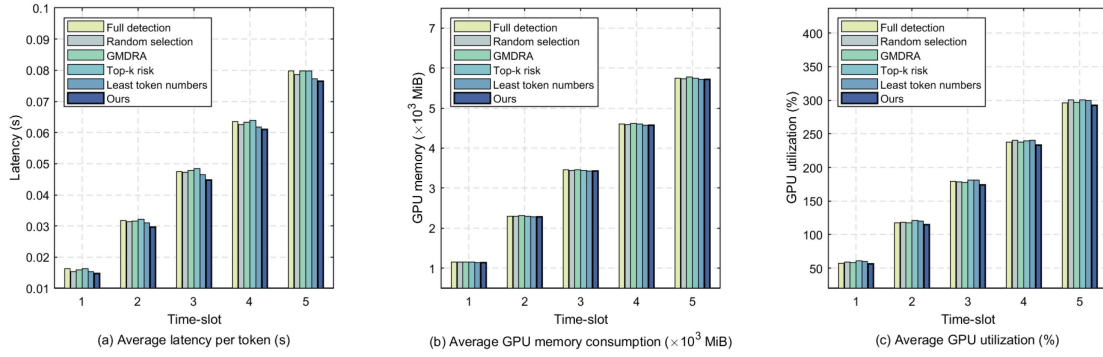


Fig. 4. Incremental comparison of average latency, GPU memory consumption, and GPU utilization across different detection strategies.

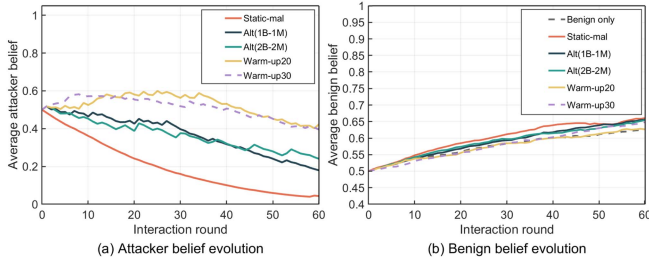


Fig. 5. Belief evolution under different attack patterns.

throughput improved from 86.97 token/s to 108.73 token/s for Qwen-4B, from 69.49 token/s to 88.44 token/s for Qwen-7B, and from 50.32 token/s to 53.13 token/s for Qwen-14B.

These results demonstrate that although large-scale models inherently introduce higher inference latency, an optimized architectural design can significantly mitigate this impact. By enabling adaptive scheduling and early detection at the edge, our framework effectively reduces redundant transmissions and balances computational load, thereby achieving higher throughput and better scalability across heterogeneous LLM environments.

C. Performance Under Different Attack Patterns

To evaluate the dynamic behavior of the belief update mechanism under complex attack behaviors, we simulate 60 rounds of continuous communication among 100 users. The experiment considers five representative attack patterns, including persistent malicious attacks, denoted as Static-mal, alternating disguise attacks, denoted as Alt(1B-1M) and Alt(2B-2M), and warm-up based attacks with different dormant lengths, denoted as Warm-up20 and Warm-up30. The first two categories correspond to persistent malicious submissions and periodic alternation between benign and malicious requests, respectively. The latter two denote attackers that remain benign during the first 20 or 30 rounds and then launch attacks in a repeating pattern of two benign rounds followed by one malicious round.

As shown in Fig. 5, belief evolution under different attack patterns is highly interpretable. Specifically, Fig. 5(a) shows that the belief values of attackers generally decrease as interactions proceed, with the fastest decline observed under persistent attacks, while the decrease is relatively slower under alternating

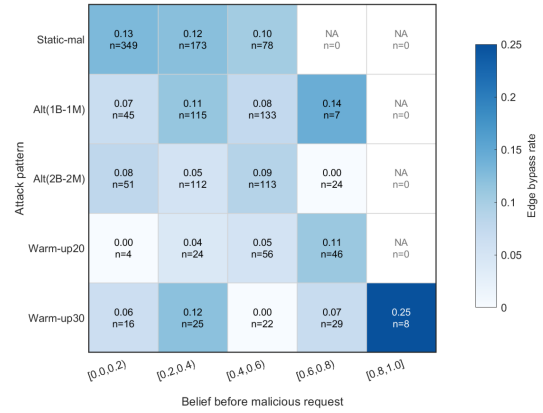


Fig. 6. Edge bypass rates of malicious requests under different prior belief intervals.

disguise and warm-up based attacks. For attacks with a dormant stage, the belief first increases due to consecutive benign interactions in the early phase and then gradually declines once malicious behavior begins to appear, indicating that the proposed mechanism can continuously correct trust after dormant attacks are activated. Meanwhile, Fig. 5(b) shows that the average belief of benign users remains steadily increasing, suggesting that the mechanism does not significantly interfere with trust accumulation for normal users. From the edge bypass results in Fig. 6, the bypass rates under all attack patterns remain generally low, and their absolute values are limited, which indicates effective coordination between edge detection and belief correction. By comparison, attack patterns with longer dormant periods exhibit relatively stronger short-term disguise capability in high-belief regions.

Overall, these results indicate that the proposed belief update mechanism maintains stable and interpretable dynamic responses under different attack patterns. It can rapidly suppress persistent attacks, sustain continuous correction under alternating disguise, and remain effective against attacks launched after a dormant stage.

D. Performance of Different Detection

To comprehensively assess the effectiveness of our adaptive detection framework, we compared five representative strategies

TABLE III
COMPARISON OF DETECTION PERFORMANCE UNDER DIFFERENT ATTACK
MODES AND STRATEGIES

Attack mode	Strategy	Acc.	Prec.	Rec.	F1
Static-mal	No detection	0.500	—	—	—
	GMDRA	0.860±0.009	0.838±0.012	0.895±0.017	0.865±0.009
	Random	0.555±0.010	0.626±0.019	0.276±0.018	0.383±0.021
	Top- <i>k</i> risk	0.857±0.026	0.871±0.014	0.838±0.043	0.854±0.029
	Ours	0.705±0.010	0.643±0.008	0.941±0.010	0.752±0.008
Alt(1B-1M)	No detection	0.500	—	—	—
	GMDRA	0.660±0.029	0.740±0.020	0.531±0.062	0.639±0.051
	Random	0.547±0.021	0.610±0.045	0.257±0.034	0.362±0.041
	Top- <i>k</i> risk	0.624±0.044	0.713±0.052	0.407±0.085	0.516±0.083
	Ours	0.683±0.010	0.643±0.014	0.865±0.017	0.692±0.018
Alt(2B-2M)	No detection	0.500	—	—	—
	GMDRA	0.677±0.039	0.754±0.035	0.523±0.072	0.616±0.062
	Random	0.566±0.010	0.645±0.017	0.293±0.020	0.403±0.022
	Top- <i>k</i> risk	0.628±0.030	0.721±0.037	0.417±0.054	0.527±0.053
	Ours	0.686±0.030	0.638±0.051	0.882±0.043	0.722±0.049
Warm-up20	No detection	0.500	—	—	—
	GMDRA	0.527±0.031	0.661±0.060	0.322±0.079	0.394±0.090
	Random	0.542±0.024	0.602±0.054	0.245±0.042	0.347±0.051
	Top- <i>k</i> risk	0.584±0.023	0.669±0.060	0.326±0.074	0.437±0.078
	Ours	0.636±0.030	0.718±0.051	0.392±0.043	0.498±0.049
Warm-up30	No detection	0.500	—	—	—
	GMDRA	0.509±0.013	0.529±0.034	0.280±0.032	0.367±0.039
	Random	0.541±0.020	0.601±0.043	0.242±0.040	0.344±0.048
	Top- <i>k</i> risk	0.574±0.023	0.671±0.041	0.294±0.042	0.407±0.047
	Ours	0.589±0.013	0.657±0.013	0.376±0.031	0.477±0.027

Note: ± denotes the standard deviation over five independent seeds.

under identical resource constraints: (i) No detection, where all prompts are directly forwarded to the LLM without any detection; (ii) GMDRA [15], a dynamic game-based detection baseline that determines prompt selection based on the optimization result and cost threshold in each round; (iii) Random selection, which randomly selects a fixed ratio of prompts for detection; (iv) Top-*k* risk, which prioritizes prompts with the highest risk scores estimated from belief updates but without contextual adaptation; and (v) Ours, which combines belief updates with BWCB scheduling for adaptive and risk-aware prompt selection. The results in this subsection are all reported on a balanced evaluation set.

As shown in Table III, the compared strategies exhibit clear differences across attack modes. In the Static-mal setting, GMDRA achieves better Accuracy and F1 score, while Top-*k* risk shows better Precision. This is reasonable because when attacks remain stable and homogeneous, threshold-based selection and belief ranking can more directly focus resources on persistently exposed high-risk users. In this setting, our method still obtains higher Recall, indicating stronger protection against missed malicious prompts.

When the attack behavior becomes non-stationary, the advantage shifts. Under Alt(1B-1 M) and Alt(2B-2 M), our method shows better overall performance in terms of Accuracy, Recall, and F1, while GMDRA maintains stronger Precision. Under Warm-up20, our method performs favorably across all four metrics. Under Warm-up30, our method achieves better Accuracy, Recall, and F1, while maintaining competitive Precision. These results indicate that once attackers switch behaviors or exploit a dormant stage to accumulate trust, methods based only on cost thresholds or historical risk ranking are more easily affected,

whereas our method can recover detection effectiveness more steadily after malicious behavior reappears.

Overall, the proposed method shows more stable overall performance across attack modes, especially in terms of Recall and F1. In particular, under alternating disguise and warm-up attacks, which are more representative of realistic deployment conditions, our method maintains better detection coverage and overall effectiveness. This suggests that a belief-driven and context-aware detection framework provides better robustness and adaptability under dynamic attacks.

E. Performance Under Continuous Workloads

To further evaluate the practical efficiency and scalability of the proposed framework, we conducted a comparative experiment across multiple detection strategies, measuring three key indicators, including average latency, GPU memory consumption, and GPU utilization, over five consecutive time-slots, as illustrated in Fig. 4. This experiment emulates long-term continuous workloads where detection requests accumulate over time, highlighting the necessity of adaptive scheduling and resource optimization. The compared methods include Full detection, Random selection, GMDRA, Top-*k* risk, Least-token selection, and Ours. These methods represent exhaustive detection, stochastic sampling, game-based scheduling, risk-driven prioritization, token-based pruning, and our belief-aware contextual scheduling strategy, respectively, forming a comprehensive comparison framework under identical experimental conditions.

As shown in Fig. 4, the proposed framework achieves the most balanced performance across all three metrics. The Full detection strategy incurs the highest latency and GPU overhead (up to 0.08 s per token and about 4.5×10^3 MiB in the final slot) due to redundant inference. Overall, these results clearly demonstrate the effectiveness of the proposed framework in handling complex and heterogeneous workloads. Compared with five representative baselines, the Ours strategy achieves comparable detection capability with significantly lower latency and resource consumption, while maintaining consistent performance under long-term continuous operation. This confirms that our design effectively harmonizes security, responsiveness, and scalability, providing a practical and efficient solution for secure large-scale LLM deployment.

F. Performance Under Different Malicious User Ratios

To comprehensively evaluate the adaptability of the proposed user-aware detection framework, we conducted parallel experiments on both the Edge Layer and the LLM Layer, as illustrated in Fig. 7 and Fig. 8. The malicious user ratio was gradually increased from 0% to 50%, and three key metrics were monitored on both sides: average latency, GPU memory consumption, and GPU utilization.

As shown in Fig. 7, the Edge Layer latency steadily increases with the number of concurrent users due to heavier workloads and communication overhead. Nevertheless, our proposed method consistently achieves the lowest latency across all settings, benefiting from Bayesian belief updating and contextual bandit-based adaptive prompt selection. On the LLM Layer,

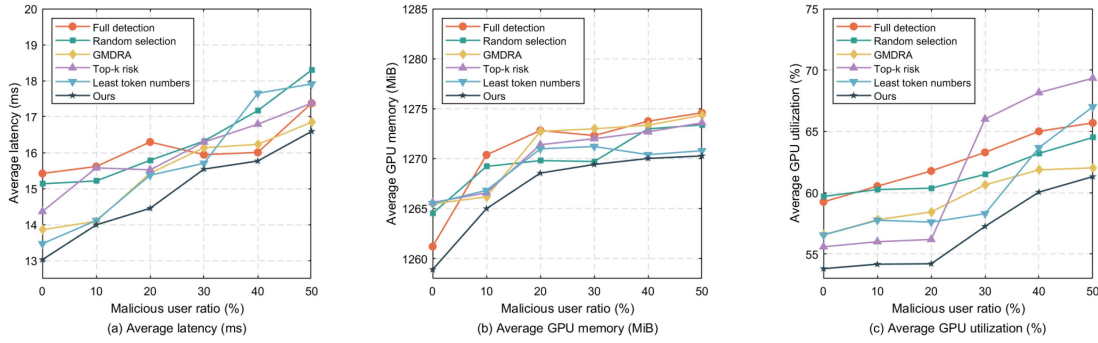


Fig. 7. Edge Layer performance variation under different malicious user ratios.

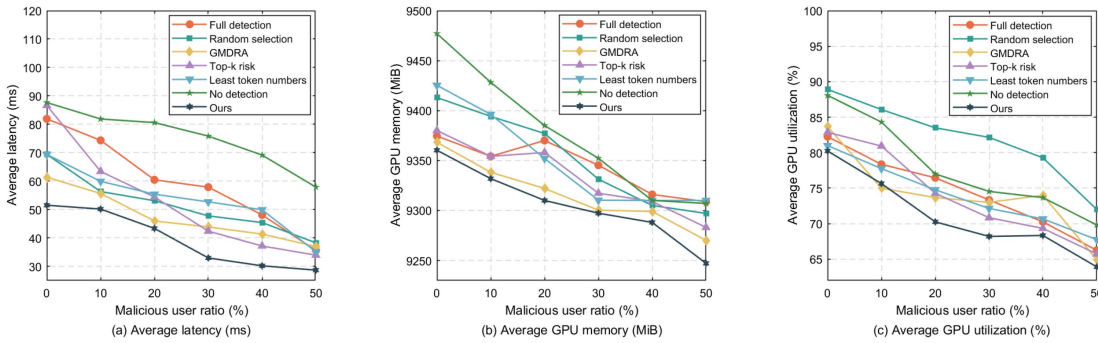


Fig. 8. LLM Layer performance under different malicious user ratios.

as shown in Fig. 8, all methods exhibit a gradual decrease in latency and resource utilization as the malicious user ratio increases. This trend occurs because higher malicious densities trigger earlier risk filtering and prompt rejection at the edge, thereby reducing the number of queries sent to the cloud. Our method further amplifies this effect by dynamically filtering high-risk prompts, leading to both lower GPU utilization and smaller memory consumption through user-aware detection prioritization. Notably, the proposed adaptive mechanism achieves comparable detection accuracy with the least computational overhead, confirming its scalability for long-term cloud-edge collaboration.

Overall, these results demonstrate that the proposed user-aware detection strategy effectively coordinates edge and cloud resources. By jointly optimizing belief-updating risk assessment and adaptive resource allocation, it achieves superior latency reduction and energy efficiency under different malicious intensities, confirming its robustness and practicality in real-world deployments.

G. Performance Under Different User Scales

To analyze the impact of user scale on system performance, we assumed that each user issued multiple requests during the experiment, and the number of concurrent users was gradually increased while keeping the malicious user ratio fixed at 10%. In this setup, the total number of requests is influenced by both the number of users and the number of requests each user makes.

The workload increases linearly with both the user count and the number of requests per user.

As shown in Fig. 9, throughput increases almost linearly when the number of users is small but gradually saturates as the user scale increases, reflecting the inference and hardware capacity limits of the LLMs. Latency also rises with user scale, especially under centralized processing architectures, such as cloud-only deployment. The proposed strategy demonstrates the best scalability by dynamically allocating detection resources across users, maintaining lower latency and higher detection accuracy even in large-scale user scenarios. Compared to GMDRA and Top- k risk strategies, our approach exhibits significant advantages in handling large-scale concurrent users through user-aware resource allocation.

In summary, despite the observed variations in throughput and latency with increasing user scale, our method successfully maintains lower latency and higher detection accuracy in large-scale environments by dynamically allocating detection resources.

H. Robustness and Adaptability Analysis

To further evaluate the robustness and adaptability of BUUAS, we conduct additional experiments on cross-dataset generalization, cross-detector robustness, and parameter sensitivity. For the cross-dataset generalization and cross-detector robustness experiments, we adopt a leave-one-dataset-out setting, where

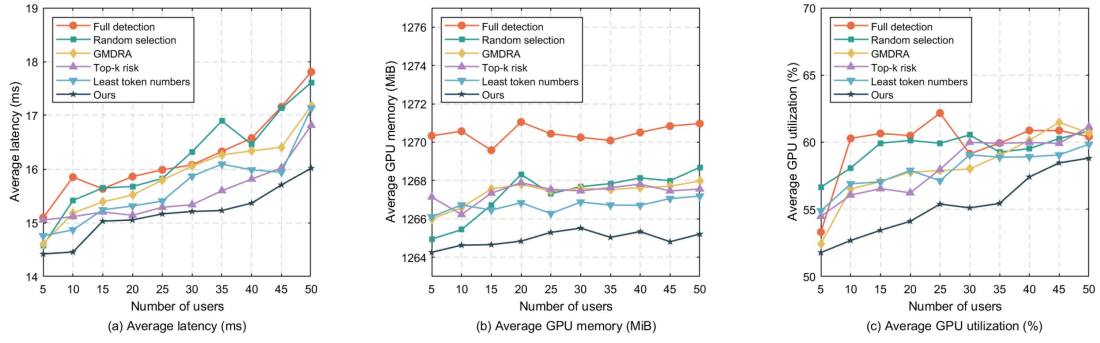


Fig. 9. LLM Layer performance under different user scales.

TABLE IV

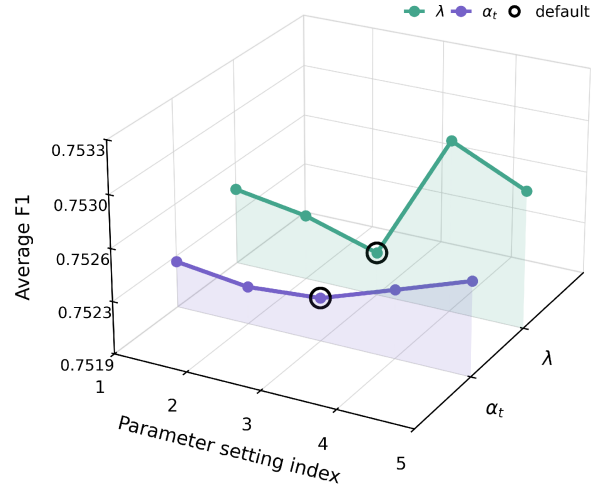
ROBUSTNESS COMPARISON UNDER DIFFERENT ATTACK MODES AND EDGE DETECTORS

Attack mode	Detector	Acc.	Prec.	Rec.	F1
Static-mal	DeBERTa	0.685±0.010	0.628±0.008	0.907±0.010	0.742±0.008
	TestSavant	0.606±0.006	0.583±0.006	0.742±0.014	0.653±0.006
	Sentinel	0.701±0.009	0.704±0.009	0.694±0.011	0.699±0.009
Alt(1B-1M)	DeBERTa	0.673±0.012	0.620±0.009	0.895±0.011	0.732±0.009
	TestSavant	0.580±0.021	0.563±0.015	0.713±0.037	0.629±0.024
	Sentinel	0.699±0.015	0.701±0.013	0.694±0.021	0.697±0.017
Alt(2B-2M)	DeBERTa	0.684±0.010	0.628±0.007	0.905±0.008	0.741±0.007
	TestSavant	0.601±0.021	0.580±0.015	0.733±0.036	0.647±0.023
	Sentinel	0.705±0.012	0.707±0.008	0.701±0.024	0.704±0.016
Warm-up20	DeBERTa	0.578±0.008	0.657±0.013	0.329±0.019	0.438±0.019
	TestSavant	0.593±0.017	0.576±0.014	0.711±0.024	0.636±0.017
	Sentinel	0.537±0.028	0.554±0.035	0.375±0.057	0.447±0.051
Warm-up30	DeBERTa	0.594±0.025	0.675±0.029	0.360±0.055	0.468±0.053
	TestSavant	0.603±0.020	0.584±0.015	0.716±0.031	0.643±0.021
	Sentinel	0.533±0.028	0.553±0.041	0.350±0.044	0.428±0.045

Note: All results are obtained under the leave-one-dataset-out setting, where Jackhhao is used only as the held-out test dataset. $\pm$ denotes the standard deviation over five independent seeds.

the vector database is constructed using the Guychuk and AhsanAyub datasets, while Jackhhao is used as an independent held-out test dataset. In the cross-detector evaluation, we compare the DeBERTa-based detector with two additional public edge-side detectors, TestSavant⁵ and Sentinel.⁶ The parameter sensitivity experiment follows the full-data evaluation setting, in which all datasets are used according to the overall experimental setup, to examine the impact of key parameter variations on BUUAS.

Table IV reports the robustness comparison under different attack modes and edge detectors. Under static and alternating attacks, Sentinel generally achieves higher Accuracy and Precision, while the DeBERTa detector obtains higher Recall and F1 scores. This indicates that different detectors exhibit different operating preferences, with Sentinel producing more conservative and precise predictions and DeBERTa providing stronger malicious-request coverage. Under warm-up attacks, TestSavant obtains higher Recall and F1 scores, while DeBERTa maintains the highest Precision. This suggests that TestSavant



Note: Setting indices 1–5 correspond to $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0\}$ and $\alpha_t \in \{0.50, 0.75, 1.00, 1.25, 1.50\}$, respectively. The default values are $\lambda = 1.0$ and $\alpha_t = 1.0$.

Fig. 10. Parameter sensitivity analysis under different key parameter settings.

behaves as a more recall-oriented detector in delayed-attack scenarios, whereas DeBERTa and Sentinel follow more conservative precision-recall trade-offs. These results demonstrate that BUUAS can adapt to detectors with different output distributions and decision characteristics, rather than relying on a specific detector backbone.

Fig. 10 presents the sensitivity analysis of λ and α_t . The average F1 score changes only slightly across different parameter settings, and no obvious performance degradation is observed. Moreover, the performance remains stable over a range of settings rather than depending on a single carefully tuned parameter value. This indicates that the effectiveness of BUUAS mainly comes from the belief-updating and BWCB-based scheduling mechanisms, rather than from parameter tuning. Therefore, BUUAS can maintain stable performance in practical deployment without complicated parameter search.

Overall, the above results show that BUUAS remains effective under unseen test data, different edge detectors, multiple attack modes, and key parameter variations. These findings verify the cross-dataset generalization ability, detector adaptability, and stability under parameter variations of BUUAS.

⁵<https://huggingface.co/testsavantai/prompt-injection-defender-base-v0>

⁶<https://huggingface.co/rogue-security/prompt-injection-jailbreak-sentinel-v2>

VI. RELATED WORK

A. Injection Attacks and Defense Mechanisms

In recent years, with the widespread deployment of large-scale LLMs, their security issues have attracted increasing attention from both academia and industry. A growing body of research has demonstrated that LLMs are particularly vulnerable to novel threats such as prompt injection attacks. To address this problem, a variety of detection and defense mechanisms have been proposed. For instance, Goyal et al. [23] introduced the LLMGuard framework, which enables real-time monitoring of unsafe behaviors during interactive use and provides a practical solution for mitigating injection risks. Rahman et al. [24] compared the performance of fine-tuned models with that of original pre-trained models for prompt injection detection, and their results indicate that fine-tuning can significantly improve detection accuracy. SecurityLingua [25] introduced a security-aware prompt compression mechanism that reduces redundant input while significantly lowering the success rate of injection attacks. More recently, Liu et al. [26] proposed the DataSentinel framework, which formulates prompt injection detection as a game-theoretic process and enhances defenses against adaptive injection attacks through adversarial fine-tuning of the detection model. RedAgent [27] further suggests that jailbreak and injection risks can be highly context-dependent in real deployments, where context-aware red teaming can generate tailored jailbreak prompts for customized LLM applications.

Beyond prompt-level defenses, some research works have explored edge-assisted security detection and collaborative frameworks to improve detection efficiency and system-level security. Zhang et al. [28] proposed AntiConcealer for reliable detection of concealed adversarial behaviors in EdgeAI-assisted IoT. Karanjai et al. [16] designed TLIESC, which integrates smart contracts with edge-based inference to prevent sensitive data from being uploaded to the cloud. Moitra et al. [17] implemented low-power adversarial detection at edge nodes, demonstrating the feasibility of lightweight edge-side defenses, while Sun et al. [21] distributed lightweight detection tasks to the edge and delegated complex analysis to the cloud, balancing latency and computational burden. These works highlight the potential of edge-cloud collaboration to mitigate prompt-based threats under realistic resource constraints.

In summary, although these studies have made important progress in prompt injection detection, most existing approaches rely heavily on centralized cloud computation. This reliance often introduces high latency and processing bottlenecks, making it difficult to satisfy the low-latency and high-reliability requirements of large-scale applications. Furthermore, current detection mechanisms generally adopt uniform strategies without adapting to diverse user or task characteristics, which limits their applicability in complex environments.

B. User-Aware Mechanisms

In the application of LLMs, user requirements and usage scenarios often vary considerably, which has motivated the recent exploration of user-aware mechanisms. Kholkar et al. [18]

proposed the CAPTURE framework, which builds a context-aware evaluation tool that directly reveals the limitations of static guardrails in dynamic settings, further underscoring the importance of incorporating dynamic and user-aware mechanisms. Almheiri et al. [19] proposed Role-aware LLMs, which emphasize differentiated security protection based on user identity and contextual information, reflecting the role of user characteristics in interactions. However, these strategies remain largely static and lack adaptability in dynamic environments. Lee et al. [20] introduced SafeRoute, which adaptively invokes guard models according to input difficulty and activates large safety models only for high-risk prompts. This design maintains accuracy while reducing overhead, but its decision criteria are limited and do not incorporate factors such as user trust or historical behavior, resulting in limited applicability in complex scenarios. Similar issues have also been discussed in privacy-preserving truth discovery over data streams [29]. More broadly, user heterogeneity extends beyond identities and service demands to the risks arising from interaction contexts and behavioral trajectories. Contextual backdoor attacks [30] show that the same model can behave differently under user-specific contexts, reinforcing the need for context- and user-aware safeguards in real deployments. Furthermore, heterogeneity also emerges at the participant level, where contributors may differ in incentives and reliability [31], motivating accountability mechanisms that record contributions and support corrective operations when needed. This issue is also reflected in the discovery of truth that preserves privacy for sparse crowd sensing, where limited participation and incomplete observations make reliable aggregation substantially more challenging [32].

In addition, Hu et al. [33] presented the QLLMS framework, which leverages quantization and heterogeneous resource scheduling to achieve lower cost and higher completion rates under partially informed environments, demonstrating that adaptive scheduling can align system performance with user-specific needs. Building on this direction, Huang et al. [15] integrated edge-cloud cooperation and Bayesian-based coordination to jointly optimize prompt security, latency, and resource utilization, revealing that user-aware strategies can be embedded in system-level collaboration to balance efficiency and security.

In summary, user-aware mechanisms have significantly advanced the adaptability and contextual sensitivity of LLM-based security systems by introducing personalized and role-specific protection strategies. Nevertheless, most existing approaches remain constrained by limited integration with adaptive scheduling and collaborative defense architectures, while still providing limited support for user-aware fine-grained decision-making. The absence of a unified framework that jointly incorporates user modeling, adaptive decision-making, and edge-cloud coordination underscores the necessity of developing belief-driven mechanisms capable of dynamically adjusting security policies in response to evolving user behaviors and system states.

VII. CONCLUSION

In this study, we propose a belief-driven adaptive security framework named BUAS for enhancing the safety and

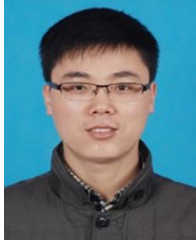
efficiency of LLM deployments in edge–cloud environments. BUUAS integrates Bayesian-inspired belief updating mechanism with a user-aware, context-adaptive scheduling mechanism, enabling user-aware risk assessment based on real-time user behavior and prompt characteristics. By intelligently allocating limited edge resources to the most security-critical inputs, BUUAS achieves a fine-grained balance among detection accuracy, responsiveness, and computational efficiency. Extensive experiments demonstrate that BUUAS significantly outperforms the compared detection strategies, showing strong adaptability under diverse workloads and varying attack intensities while maintaining low latency and resource consumption. Overall, BUUAS establishes a user-aware, dynamic, and security-oriented paradigm for scalable and trustworthy large-scale language model deployment at the network edge.

REFERENCES

- [1] H. Shao et al., “LMDrive: Closed-loop end-to-end driving with large language models,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 15120–15130.
- [2] K. Singhal et al., “Large language models encode clinical knowledge,” *Nature*, vol. 620, no. 7972, pp. 172–180, 2023.
- [3] K. Malinka, M. Peresini, A. Firc, O. Hujnák, and F. Janus, “On the educational impact of ChatGPT: Is artificial intelligence ready to obtain a university degree,” in *Proc. Conf. Innov. Technol. Comput. Sci. Educ. V. 1*, 2023, pp. 47–53.
- [4] W. Chen, Y. Ji, Z. Chen, J. Xu, and Z. Liu, “LLMGR: Large language model-based generative retrieval in Alipay search,” in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2024, pp. 2847–2851.
- [5] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, “A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly,” *High-Confidence Comput.*, vol. 4, no. 2, 2024, Art no. 100211.
- [6] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proc. 33rd USENIX Secur. Symp.*, 2024, vol. 24, pp. 1831–1847.
- [7] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, “SecAlign: Defending against prompt injection with preference optimization,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2025, pp. 2833–2847.
- [8] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “StruQ: Defending against prompt injection with structured queries,” in *Proc. 34th USENIX Secur. Symp. (USENIX Secur. 25)*, 2025, pp. 2383–2400.
- [9] C. Zhang, M. Yu, W. Wang, and F. Yan, “Enabling cost-effective, SLO-aware machine learning inference serving on public cloud,” *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1765–1779, Jul.–Sep. 2020.
- [10] P. Patel et al., “Characterizing power management opportunities for LLMs in the cloud,” in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2024, vol. 3, pp. 207–222.
- [11] X. Chen, J. Cao, Y. Sahni, M. Zhang, Z. Liang, and L. Yang, “Mobility-aware dependent task offloading in edge computing: A digital twin-assisted reinforcement learning approach,” *IEEE Trans. Mobile Comput.*, vol. 24, no. 4, pp. 2979–2994, Apr. 2024.
- [12] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, “EdgeShard: Efficient LLM inference via collaborative edge computing,” *IEEE Internet Things J.*, vol. 12, no. 10, pp. 13119–13131, 2025.
- [13] P. Zhang, X. Cheng, Z. Zhang, Y. Zhu, and J. Zhang, “Maximizing area coverage in privacy-preserving worker recruitment: A prior knowledge-enhanced geo-indistinguishable approach,” *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 5138–5151, 2025.
- [14] X. Luo, T. Yu, and X. Xiao, “Prompt inference attack on distributed large language model inference frameworks,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2025, pp. 1739–1753.
- [15] H. Huang, T. Meng, and W. Jia, “Joint optimization of prompt security and system performance in edge-cloud LLM systems,” in *Proc. IEEE INFOCOM -IEEE Conf. Comput. Commun.*, 2025, pp. 1–10.
- [16] R. Karanjai and W. Shi, “Trusted LLM inference on the edge with smart contracts,” in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency*, 2024, pp. 1–7.
- [17] A. Moitra, A. Bhattacharjee, Y. Kim, and P. Panda, “RobustEdge: Low power adversarial detection for cloud-edge systems,” *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 2, pp. 2101–2111, Apr. 2024.
- [18] G. Kholkar and R. Ahuja, “CAPTURE: Context-aware prompt injection testing and robustness enhancement,” in *Proc. First Workshop LLM Secur.*, 2025, pp. 176–188.
- [19] S. Almheiri et al., “Role-aware language models for secure and contextualized access control in organizations,” in *Proc. Int. Joint Conf. Natural Lang. Process. Asia-Pacific Chapter Assoc. Comput. Linguistics*, 2025, pp. 490–511.
- [20] S. Lee et al., “SafeRoute: Adaptive model selection for efficient and accurate safety guardrails in large language models,” in *Proc. Findings Assoc. Comput. Linguist.*, 2025, pp. 2053–2069.
- [21] G. Sun and J. Zhang, “A distributed intrusion detection framework for cloud and edge computing-assisted Internet of Things,” in *Proc. Int. Conf. Comput. Sci. Comput. Intell.*, 2024, pp. 34–47.
- [22] P. He, J. Gao, and W. Chen, “DeBERTaV3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing,” in *Proc. 11th Int. Conf. Learn. Representations*, Kigali, Rwanda, May 2023, pp. 23017–23032.
- [23] S. Goyal et al., “LLMGuard: Guarding against unsafe LLM behavior,” in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 21, pp. 23790–23792.
- [24] M. A. Rahman et al., “Fine-tuned large language models (LLMs): Improved prompt injection attacks detection,” in *Proc. IEEE 49th Annu. Comput., Software, Appl. Conf.*, 2025, pp. 1033–1039.
- [25] Y. Li, S. Ahn, H. Jiang, A. H. Abdi, Y. Yang, and L. Qiu, “SecurityLingua: Efficient defense of LLM jailbreak attacks via security-aware prompt compression,” in *Proc. 2nd Conf. Lang. Model.*, 2025, pp. 123–132.
- [26] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, “DataSentinel: A game-theoretic detection of prompt injection attacks,” in *Proc. IEEE Symp. Secur. Privacy*, 2025, pp. 2190–2208.
- [27] H. Xu et al., “RedAgent: An autonomous agent for context-aware red teaming of LLM jailbreaks,” *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 3, pp. 6506–6521, May/Jun. 2026.
- [28] J. Zhang et al., “AntiConcealer: Reliable detection of adversary concealed behaviors in edgeai-assisted IoT,” *IEEE Internet Things J.*, vol. 9, no. 22, pp. 22184–22193, Nov. 2022.
- [29] P. Zhang, Z. Zhang, Y. Cao, S. Wang, X. Cheng, and J. Zhang, “Locally differentially private truth discovery over data streams,” *IEEE Trans. Mobile Comput.*, vol. 25, no. 6, pp. 7698–7715, Jun. 2026.
- [30] A. Liu et al., “Compromising LLM driven embodied agents with contextual backdoor attacks,” *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 3979–3994, 2025.
- [31] X. Zuo et al., “Federated trustchain: Blockchain-enhanced LLM training and unlearning,” *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 3, pp. 6457–6473, May/Jun. 2026.
- [32] P. Zhang et al., “Locally differentially private truth discovery for sparse crowdsensing,” *IEEE Trans. Knowl. Data Eng.*, vol. 38, no. 2, pp. 1189–1205, Feb. 2026.
- [33] M. Hu, Q. He, and D. Wu, “QLLMs: Quantization-adaptive LLM scheduling for partially informed edge serving systems,” in *Proc. IEEE INFOCOM -IEEE Conf. Comput. Commun.*, 2025, pp. 1–10.



Wenjing Chen is currently working toward the PhD degree with the School of Computer Science and Technology, Anhui University. Her research interests include computer networks, IoT security, and large language model (LLM) security.



Jie Cui (Senior Member, IEEE) was born in Henan Province, China, in 1980. He received the PhD degree from the University of Science and Technology of China, in 2012. He is currently a professor and PhD supervisor with the School of Computer Science and Technology, Anhui University. He has more than 150 scientific publications in reputable journals such as *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Mobile Computing*,

IEEE Transactions on Parallel and Distributed Systems, *IEEE Transactions on Computers*, *IEEE Transactions on Vehicular Technology*, *IEEE Transactions on Intelligent Transportation Systems*, *IEEE Transactions on Network and Service Management*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Industrial Electronics*, *IEEE Transactions on Cloud Computing*, and *IEEE Transactions on Multimedia*, academic books and international conferences. His research interests include applied cryptography, IoT security, vehicular ad hoc network, cloud computing security and software-defined networking (SDN).



Wenjie Huang (Member, IEEE) received the BS degree from the Chongqing University of Posts and Telecommunications and the PhD degree in computer science and technology from the University of Electronic Science and Technology of China. He is currently a lecturer with the School of Computer Science and Technology, Anhui University, China. His research interests include edge computing, mobile computing and wireless.



Jing Zhang was born in Henan Province, China, in 1990. She received the MA Eng and PhD degrees in computer science from Anhui University, in 2021. She is currently an associate professor with School of Computer Science and Technology, Anhui University. She has nearly 20 scientific publications in reputable journals (such as *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Vehicular Technology*, *IEEE Transactions on Intelligent Transportation Systems*, *Information Sciences*, *Science China Information Sciences*, and *Vehicular Communications*) and international conferences. Her research interests include vehicular ad hoc network, IoT security and applied cryptography.



Lu Wei was born in Anhui Province, China, in 1993. He received the PhD degree in computer science from Anhui University, in 2022. He is currently a lecture with School of Computer Science and Technology, Anhui University. He has more than 10 scientific publications in reputable journals such as *IEEE Transactions on Information Forensics and Security*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Intelligent Transportation Systems*. His research interests include vehicular ad hoc networks and applied cryptography.



Geyong Min (Senior Member, IEEE) received the BSc degree in computer science from Huazhong University of Science and Technology, China, in 1995 and the PhD degree in computing science from the University of Glasgow, U.K., in 2003. He is currently a professor of High Performance Computing and Networking with the Department of Computer Science, the College of Engineering, Mathematics and Physical Sciences, the University of Exeter, U.K. His research interests include computer networks, wireless communications, parallel and distributed computing, ubiquitous computing, multimedia systems, modelling and performance engineering.